

# md2D / md3D User Manual

md2D and md3D compute the light diffracted by periodic or rough structures with the rigorous differential method. This manual covers what the programs compute, how to describe a structure, how to read the results and how to choose the numerical parameters.

Manual version 1 · September 2026 · Code as in branch `thesis-validation-reproduction`

1. Overview

2. Building and running

3. Geometry and conventions

4. Parameter file

5. Structure files

6. Result files

7. Numerical parameters and convergence

8. Limitations and known issues

9. Validation

10. References

## 1 Overview

Both programs solve Maxwell's equations in Fourier space inside the modulated region of a structure and match the solution to plane-wave (Rayleigh) expansions in the two homogeneous half-spaces around it. The only approximations are the truncation of the Fourier series and the discretisation along  $z$ . Four ingredients make the method stable, accurate and fast:

- **S-matrix propagation** through `NS` slices, which removes the numerical instability of thick structures [Li 1996b].
- **Fast Fourier factorisation** with a normal-vector field (Li's rules as extended by Popov and Nevière), which gives fast convergence in TM polarisation and for metals [Li 1996a; Popov & Nevière 2000, 2001].
- **Matrix Runge–Kutta integration**: the propagation matrix of each slice is built directly by an order-4 Runge–Kutta scheme, instead of integrating once per unknown as the classical shooting method did. This makes the code tens to hundreds of times faster.
- **A z-invariant mode** ( `Z_INVAR` ) for binary structures, equivalent to RCWA / the Fourier modal method.

| PROGRAM | STRUCTURE                                              | ILLUMINATION                                                  | STRUCTURE DESCRIPTIONS                                                 |
|---------|--------------------------------------------------------|---------------------------------------------------------------|------------------------------------------------------------------------|
| md2D    | invariant along $y$ , periodic along $x$ (period $L$ ) | plane of incidence $\perp$ grooves, TE or TM                  | surface $h(x)$ , stack of interfaces, index map $n(x,z)$               |
| md3D    | periodic along $x$ and $y$ ( $L_x, L_y$ )              | any $\theta$ , azimuth $\varphi$ , linear polarisation $\psi$ | surface $h(x,y)$ , stack of interfaces, z-invariant index map $n(x,y)$ |

Conical incidence on a one-dimensional grating is computed with `md3D` and `Ny = 0`. A non-periodic object (rough surface, isolated defect) is computed as one period of a large artificial period  $L$ . The Fourier discretisation makes every structure periodic, and the scattered intensity is recovered from the efficiencies (§ 6).

The code comes from L. Arnaud's PhD work at Institut Fresnel (Marseille, 2004–2008; thesis in French [Arnaud 2008]), with later additions up to 2019. The thesis chapters used here are 1 (classical formulation), 2 (stable algorithms and validation), 3 (conical incidence) and 4 (three-dimensional method). Part 2 of the thesis contains applications: scatterometry, rough surfaces, rough gratings.

## 2 Building and running

Requirements: a C compiler (gcc), FFTW 3, BLAS and LAPACK. On Debian or Ubuntu: `apt install build-essential libfftw3-dev libblas-dev liblapack-dev`. Also install OpenBLAS ( `apt install libopenblas0-pthread` ): it replaces the reference BLAS without any rebuild and makes both programs 3–8 times faster.

```
make -C md2D          # builds md2D/md2D
make -C md3D          # builds md3D/md3D
make -C utils profilGen lire_tab lire_tab2 refractive_index  # optional helpers (no default target)
```

Library choices are switches at the top of the `md2D` and `md3D` Makefiles ( `HAVE_BLAS` , `HAVE_LAPACK` , `HAVE_MKL` , `HAVE_ATLAS` ). The default uses plain BLAS/LAPACK.

### Running

```
md2D -param my_param.txt          # default: md2D_param.txt in the current directory
md2D -param my_param.txt -N 60 -pola TM -lambda 0.55
md3D -param my_param.txt -Nx 8 -Ny 8 -psi 90
```

Any parameter can be given on the command line as `-key value` , and the command line wins over the file. Complex indices are split on the command line: `-n_sub_re 1.3 -n_sub_im 7.1` for `md2D`, `-nu_sub_re / -nu_sub_im` for `md3D`. This makes scans easy to script (Practical guide).

Results go to `<profile_name>.txt` in the *current directory*, and an existing file with that name is overwritten. The terminal shows progress and a summary. `verbosity 0, 1` or `2` controls the amount.

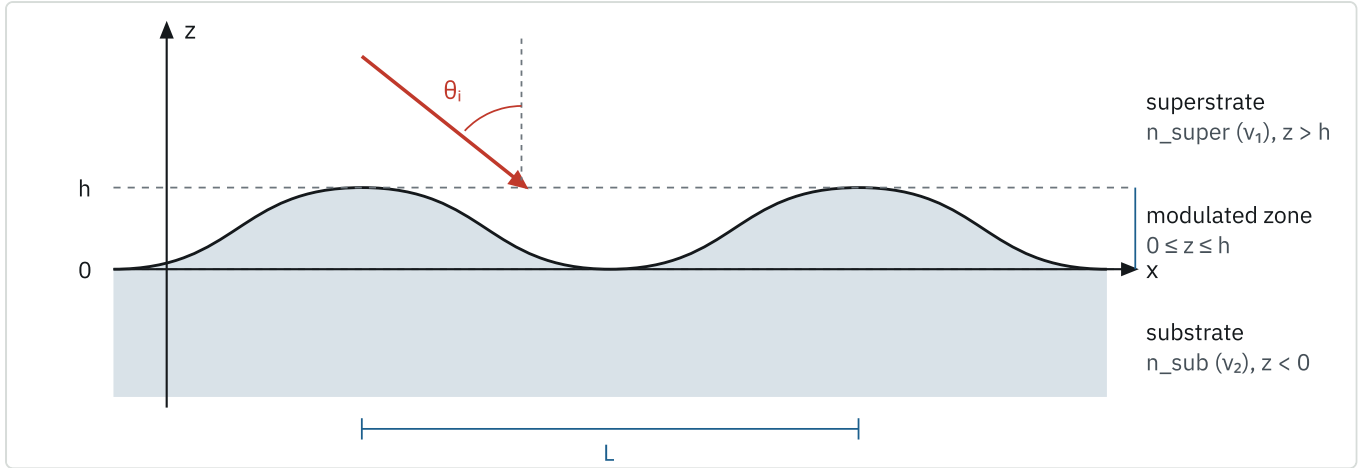
### A first run

Aluminium sinusoidal grating of Tayeb (1994), TE, normal incidence. Copy the profile file `md2D/sin_2048.txt` , which ships with the code, next to the parameter file.

```
# tayeb_alu.txt
n_super = 1.0 + i0.0
n_sub   = 1.3 + i7.1      # aluminium at 0.6 µm
L       = 0.8333333
h       = 0.8
lambda  = 0.6
theta_i = 0.0
pola    = TE
i_field_mode = PLANE_WAVE
calcul_type = STD
calcul_method = RK4
fft_filter = NONE
smoothing  = 0
N         = 40
NS        = 1000
delta_h   = 0.0001      # required, not used
profile_name = alu_TE ; profile_file = sin_2048.txt
```

Run `md2D -param tayeb_alu.txt` ; it takes about a minute. The file `alu_TE.txt` then contains `eff_r = 0.31553 0.17990 0.31553` for orders  $-1, 0, +1$ , against  $0.31558 / 0.18016$  from the method of fictitious sources.

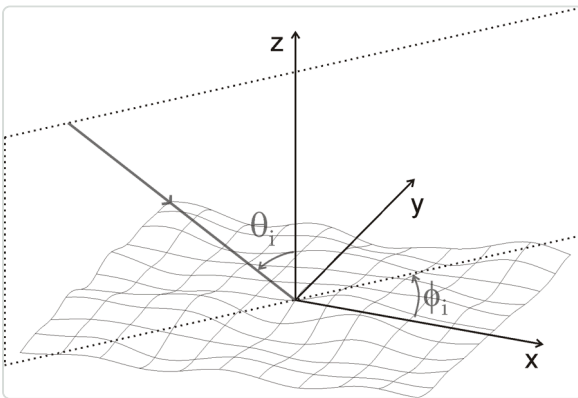
### 3 Geometry and conventions



md2D geometry. The structure is invariant along  $y$ ; the wave comes from the superstrate and propagates towards  $-z$ . The material below the profile line is the substrate material, unless a stack or an index map says otherwise.

- **Units.** Any length unit may be used, as long as `lambda`, `L / Lx / Ly` and `h` use the same one. Angles are in degrees.
- **Refractive indices** are complex,  $v = n + i\kappa$ . An absorbing medium has  $\kappa > 0$ : aluminium at  $0.6 \mu\text{m}$  is  $1.3 + i7.1$ . Write them `a + ib` or `a - ib`, with or without spaces; `a + bi`, `a + b*i` and a plain real number (`1.5`) are accepted too. The same forms work wherever a complex value is read: parameter files, structure files and the command line.
- **Diffraction orders.** With  $v_1$  the superstrate index, order  $n$  has the tangential wave number  $\sigma_n = k_0 v_1 \sin \theta_i + n 2\pi/L$ . Its direction satisfies  $v \sin \theta_n = v_1 \sin \theta_i + n\lambda/L$ , with  $v$  the index of the medium it propagates in. The specular order 0 comes out at  $\theta_0 = \theta_i$ . In md3D the orders are pairs  $(n_x, n_y)$  with  $\sigma_x = k_0 v_1 \sin \theta \cos \varphi + n_x 2\pi/L_x$  and  $\sigma_y = k_0 v_1 \sin \theta \sin \varphi + n_y 2\pi/L_y$ .
- **Efficiency** of an order is the flux of its Poynting vector through a plane  $z = \text{const.}$ , divided by the incident flux. For lossless media the reflected and transmitted efficiencies add up to 1.

#### Incidence and polarisation in md3D

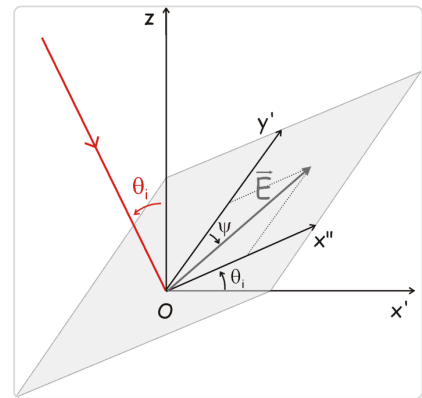


Axes of md3D.  $\phi_i$  is the angle between the plane of incidence and  $(Ox)$ ;  $\theta_i$  is measured from  $(Oz)$ . From [Arnaud 2008], chap. 4.

The incident field is

$$\mathbf{E}_i = (\cos \theta \cos \varphi \sin \psi - \sin \varphi \cos \psi) \mathbf{e}_x + (\cos \varphi \cos \psi + \cos \theta \sin \varphi \sin \psi) \mathbf{e}_y + \sin \theta \sin \psi \mathbf{e}_z.$$

md2D has no `phi_i` or `psi`. Its polarisation is set by `pol_a = TE` ( $\mathbf{E} \parallel y$ ) or `pol_a = TM` ( $\mathbf{H} \parallel y$ ).



$\psi$  is the angle between  $\mathbf{E}_i$  and the normal  $(Oy')$  to the plane of incidence:  $\psi = 0^\circ$  is TE (s),  $\psi = 90^\circ$  is TM (p). From [Arnaud 2008], chap. 3.

## 4 Parameter file

A parameter file is plain text with lines `key = value`. Several pairs may share a line when separated by `;`. Text after `#` is a comment. If a key appears twice, the first occurrence wins. Keys are case-sensitive.

**Unknown keys are silently ignored.** A typo such as `theta = 30` leaves `theta_i` missing (an error) or, for optional keys, at its default value. The parameter summary printed with `verbosity = 2` shows what was actually read.

### md2D keys

| KEY                                       |                  | VALUES                                                               | MEANING                                                                                                                                            |
|-------------------------------------------|------------------|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>profile_file</code>                 | <code>req</code> | path                                                                 | structure file (§ 5)                                                                                                                               |
| <code>profile_name</code>                 | <code>req</code> | text                                                                 | results are written to <code>&lt;profile_name&gt;.txt</code>                                                                                       |
| <code>lambda</code> , <code>L</code>      | <code>req</code> | length                                                               | wavelength in vacuum, period                                                                                                                       |
| <code>h</code>                            | <code>req</code> | length or <code>AUTO</code>                                          | thickness of the modulated zone; the profile is rescaled to <code>[0, h]</code> . <code>AUTO</code> keeps the height found in the file             |
| <code>theta_i</code>                      | <code>req</code> | degrees                                                              | angle of incidence                                                                                                                                 |
| <code>pola</code>                         | <code>req</code> | <code>TE</code>   <code>TM</code>                                    | polarisation                                                                                                                                       |
| <code>n_super</code> , <code>n_sub</code> | <code>req</code> | complex                                                              | superstrate and substrate indices                                                                                                                  |
| <code>N</code>                            | <code>req</code> | integer or <code>AUTO</code>                                         | truncation: orders <code>-N...N</code> are kept ( <code>2N+1</code> harmonics)                                                                     |
| <code>NS</code>                           | <code>req</code> | integer or <code>AUTO</code>                                         | number of S-matrix slices = number of integration steps                                                                                            |
| <code>calcul_type</code>                  | <code>req</code> | <code>STD</code>   <code>NEAR_FIELD</code>                           | efficiencies; or efficiencies plus field maps. <code>GUIDED</code> , <code>SWIFTS</code> , <code>I_SWIFTS</code> are undocumented research modes   |
| <code>calcul_method</code>                | <code>req</code> | <code>RK4</code>   <code>Z_INVAR</code>   <code>IMPROVED_RCWA</code> | integration scheme (§ 7). <code>IMPLICIT_RK</code> , <code>EULER</code> , <code>EULER2</code> exist but are not validated                          |
| <code>i_field_mode</code>                 | <code>req</code> | <code>PLANE_WAVE</code>                                              | only plane waves are implemented                                                                                                                   |
| <code>fft_filter</code>                   | <code>req</code> | <code>NONE</code>   <code>HAMMING</code>                             | optional Hamming window on the Fourier coefficients of the permittivity; applied in <code>TM</code> only                                           |
| <code>delta_h</code>                      | <code>req</code> | length                                                               | <b>read but not used</b> (one integration step per slice)                                                                                          |
| <code>smoothing</code>                    | <code>opt</code> | <code>0</code>   <code>1</code> , default <code>0</code>             | <code>1</code> adds a transition layer of thickness <code>1_smooth</code> that smooths the permittivity at interfaces (convergence aid for metals) |
| <code>verbosity</code>                    | <code>opt</code> | <code>0</code>   <code>1</code>   <code>2</code>                     | terminal output, default <code>2</code>                                                                                                            |
| <code>sigma0_normed</code>                | <code>opt</code> | complex                                                              | $\sigma_0/k_0$ ; replaces <code>theta_i</code> and allows an evanescent incident wave ( $ \sigma_0/k_0  > v_1$ )                                   |
| <code>imposed_S_steps</code>              | <code>opt</code> | <code>AUTO</code>   <code>FROM_FILE</code>                           | put slice boundaries on horizontal interfaces (multilayers); with <code>hmin_Sstep</code> , <code>imposed_S_steps_filename</code>                  |

### md3D keys

md3D uses `nu_...` instead of `n_...` for indices and has no `pola`, `fft_filter` or `smoothing`.

| KEY                         |     | VALUES            | MEANING                                                  |
|-----------------------------|-----|-------------------|----------------------------------------------------------|
| profile_file , profile_name | req | path, text        | as in md2D                                               |
| lambda , Lx , Ly            | req | length            | wavelength, periods. With $N_y = 0$ , $L_y$ is arbitrary |
| h                           | req | length or AUTO    | thickness of the modulated zone                          |
| coef_h                      | opt | number, default 1 | multiplies h and the profile heights                     |
| theta_i , phi_i , psi       | req | degrees           | incidence, azimuth, polarisation (§ 3)                   |
| nu_super , nu_sub           | req | complex           | superstrate and substrate indices                        |
| Nx , Ny                     | req | integers          | truncation along x and y: $(2N_x+1)(2N_y+1)$ harmonics   |
| NS                          | req | integer or AUTO   | number of slices = integration steps                     |
| calcul_type                 | req | STD               | only mode                                                |
| calcul_method               | req | RK4   Z_INVAR     | Z_INVAR for z-invariant index maps                       |
| i_field_mode                | req | PLANE_WAVE        |                                                          |
| delta_h                     | req | length or AUTO    | not used (a warning says so)                             |
| verbosity                   | opt | 0   1   2         | default 2                                                |

**AUTO values** (both programs):  $NS = \lceil 5 \cdot h \cdot |v_{\text{sub}}| / \lambda \rceil$  , i.e. slices of about  $\lambda/(5|v|)$ . This is enough for rough work on dielectrics and too coarse for metals in TM.  $N = \text{AUTO}$  (md2D) keeps all propagating orders plus  $\max(3, 10 \%)$  evanescent ones.

## 5 Structure files

A structure file starts with a few `key = value` lines, followed by a labelled array of numbers separated by blanks or line breaks. The heights in a profile are shifted and scaled to span  $[0, h]$  , so only the shape matters. The optional keys `h_min` / `h_max` in the file extend the modulated zone beyond the profile, which is useful for near-field maps.

### md2D

**H\_X** One interface  $z = h(x)$  between superstrate and substrate, sampled at  $N_x$  equidistant points over one period.

```
type_profil = H_X
N_x = 2048
profil =
0.500000
0.501534
...
```

**MULTICOUCHES** A stack of  $N_{\text{layers}}$  materials bounded by  $N_{\text{layers}} + 1$  interfaces. There is one line per x point and one column per interface, ordered from top to bottom. Layer  $j$  , of index  $n_j$  , lies between columns  $j$  and  $j+1$  ; the superstrate is above column 1 and the substrate below the last column. Interfaces may touch (zero thickness) but must not cross. Example: a lamellar line of index  $n_1$  on the substrate (`profilGen CARRE02_B`):

```
N_x = 1024
type_profil = MULTICOUCHES
N_layers = 1
n1 = 9.128 + i66.433
profil =
0.2 0.2      # no line here: layer 1 has zero thickness
...
0.2 0.0      # line of material n1 from z = 0 to 0.2
...
```

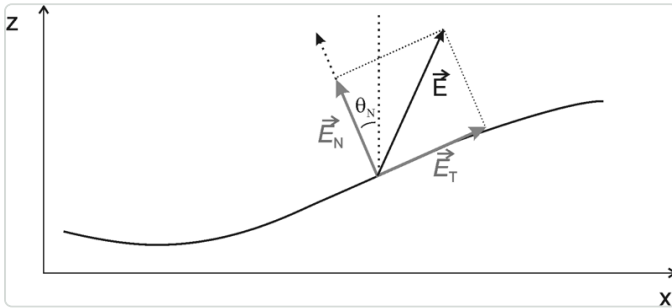
**N\_XYZ** Index map with  $N_x$  ,  $N_z$  and arrays `Re_n_xyz` and (optionally) `Im_n_xyz` , row by row from the top ( $z = h$ ) down, x varying fastest. **TE only**: no normal-vector field is defined for

this type.

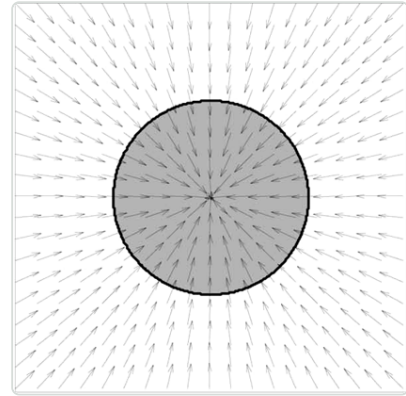
The normal vector needed in TM is computed automatically from  $h(x)$  by centred finite differences. For stacks it is interpolated between the interfaces.

## md3D

- H\_XY** One interface  $z = h(x,y)$ : `profile_type = H_XY`, `Nprx`, `Npry`, then `profil =` followed by `Npry` rows of `Nprx` values ( $x$  along a row). The normal field is computed automatically, so no separate normal-vector file is needed. `Npry = 1` describes a 1D profile, used for conical incidence with  $N_y = 0$ .
- MULTICOUCHES** As in md2D, with one line per  $(x, y)$  point ( $x$  fastest), `Nprx`, `Npry`, `N_layers`, `n1 ...`
- N\_XY\_ZINVAR**  $z$ -invariant index map (holes, pillars, patterned layer of thickness  $h$ ). The file lists indices `n1 = ...`, `n2 = ...`, then `profile_type = N_XY_ZINVAR`, `Nprx`, `Npry`, `Nprz = 1`, then `n_xyz =` followed by an integer map (1, 2, ...) referring to those indices. `Re_n_xyz` / `Im_n_xyz` arrays can be given instead. Use `calcul_method = Z_INVAR`; `NS = 1` is enough for a thin layer (see the slice rule in § 7).



Normal/tangential decomposition that underlies the fast factorisation. The normal must be defined everywhere in the modulated zone. From [Arnaud 2008], chap. 2.



For `N_XY_ZINVAR` md3D uses a radial normal field centred on the cell. It suits a centred round inclusion [Schuster 2007] and nothing else (§ 8).

`N_XYZ` (non-invariant 3D index map) and `H_XY_plus_STACK` are parsed by md3D but have no validated normal field or energy balance; treat them as experimental.

## Generating structure files

- `utils/profilGen TYPE [options] > file` writes md2D profiles: `SINUS01`, `SINUS02`, `CARRE01 / 02 / 03` (lamellar), `RECTANGLE04`, `TRAPEZ01 / 02`, `FINITE_GRATING_B`, `ALEAT01` (random rough profile), `ORIGAMI` ... The number of points is set with `-N_profil`, geometry with `-L1 -h1 -h2 -alpha` ...
- `validation_cases/tools/mkprofile.py` (Python, no dependencies) writes an echelette (md2D), a rectangular pyramid (md3D `H_XY`) and a circular hole map (md3D `N_XY_ZINVAR`).
- Any script that prints the format above will do. Sample profiles are in `md2D/` (`sin_2048.txt`, `cos_256.txt`, `sinLayer_2048.txt`) and `md3D/` (`pr_sin3D_256x256.txt`, `carre_256_1D.txt`).

## 6 Result files

The result file repeats the parameters, then lists the results as labelled arrays:

| LABEL                                                                                     | CONTENT                                                                                                                              |
|-------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <code>sum_eff</code> , <code>one_minus_sum_eff</code>                                     | total efficiency and its departure from 1 (absorption, or numerical error in lossless cases)                                         |
| <code>somm_eff_r</code> , <code>somm_eff_t</code>                                         | total reflected and transmitted efficiency                                                                                           |
| <code>N_eff_r</code> , <code>theta_r</code> , <code>eff_r</code> , <code>arg_Ar</code>    | md2D, reflected propagating orders: order number, angle (deg), efficiency, phase of the amplitude (rad)                              |
| <code>N_eff_t</code> , <code>theta_t</code> , <code>eff_t</code> , <code>arg_At</code>    | md2D, same for transmission                                                                                                          |
| <code>eff_r</code> , <code>nx_eff_r</code> , <code>ny_eff_r</code> (and <code>_t</code> ) | md3D: 2D tables, one printed row per $n_y$ . Entries <code>inf</code> mark evanescent orders inside the rectangle of printed indices |

- **Absorbing substrate.** With a metal substrate, `eff_t` and `theta_t` are not meaningful (the transmitted waves are evanescent, and `nan` may appear). Use the reflected efficiencies; the absorption is  $1 - \Sigma e_r$ .
- **Extracting values:** `cat res.txt | utils/lire_tab eff_r` prints one array, and `validation_cases/tools/mdres.py` reads an efficiency by order from Python.
- **Non-periodic structures.** For a rough surface computed with an artificial period  $L$ , the scattered intensity per unit angle is  $dI/d\theta(\theta_n) = (L/\lambda) v \cos \theta_n e_n$ , with  $v$  the index of the medium of observation [Arnaud 2008, § 1.6].
- **Near field** (`calcul_type = NEAR_FIELD`, md2D). Besides the result file, the program prints field maps to the terminal on an  $NS \times N_x$  grid: `ReEx ImEx ReEz ImEz ReHpy ImHpy` in TM, `ReEy ImEy ReHpx ImHpx ReHpy ImHpy` in TE. `Hp` is the magnetic field scaled by the code's normalisation. Redirect the output to a file and extract each map with `utils/lire_tab2 ReEx`. A complete example is in `Applications/ExempleUtilisationNearField/`.

## 7 Numerical parameters and convergence

Three numbers control accuracy and cost: the truncation `N` (or `Nx` , `Ny` ), the number of slices `NS` , and the sampling of the structure file. In the current code each S-matrix slice is integrated by exactly one Runge–Kutta step. The thesis's separate integration step  $\Delta h$  is therefore  $h/NS$ , and `delta_h` is ignored.

| PARAMETER       | EFFECT                                                                                    | GUIDANCE (FROM THE THESIS AND THE 2026 REPRODUCTION)                                                                                                                                                                                                                                     |
|-----------------|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>N</code>  | md2D cost $\propto N^3 \cdot NS$ ; md3D cost $\propto ((2N_x+1)(2N_y+1))^3 \cdot NS$      | Dielectric gratings: errors below $10^{-5}$ once all propagating orders plus a few evanescent ones are kept. Metals: $\approx 10^{-3}$ at $N = 20$ , $\approx 10^{-4}$ at $N = 40-50$ . <b>Energy conservation does not detect a too-small N</b> , so always compare two values of $N$ . |
| <code>NS</code> | cost $\propto NS$ ; errors of the RK4 scheme                                              | Dielectric, $h/NS = \lambda/200$ : $\approx 10^{-5}$ ; $\lambda/2000$ : $\approx 10^{-6}$ ; $\lambda/10\,000$ : $\approx 10^{-8}$ . Metal in TM: $h/NS \leq \lambda/700$ ( $NS = 100$ diverged for $h = 1.3\lambda$ ). $1 - \Sigma e$ is a good indicator here.                          |
| profile points  | the Fourier coefficients of the permittivity and the normal are computed from the samples | Use many more points than harmonics. 256 points at $N = 100$ cost 2 % on weak orders; with 2048 points the results agreed with an independent method to $10^{-8}$ . For md3D, 256–512 points per side.                                                                                   |

**Slice boundaries.** Each slice samples the structure from its own side: at its bottom as the limit from above, at its top as the limit from below. So when an interface lies exactly on a slice boundary (flat layers, lamellar gratings, layered index maps), a flat interface is exact at any `NS` and layered structures converge at fourth order.

### Choosing `calcul_method`

- `RK4` is the general method for any profile, and the one to use for curved or sloped interfaces.
- `Z_INVAR` treats each slice as z-invariant and propagates it exactly, as RCWA / the Fourier modal method does. It is exact and fast for binary structures (lamellar gratings, holes), but a single slice must stay thin: inside a slice of thickness  $t$  the evanescent orders grow like  $e^{2\pi N t/L}$ , which overflows for thick layers at large  $N$  ( $NS = 1$  failed for  $h = 0.23 L$  at  $N = 32$ ). A safe rule is  **$NS \geq N \cdot h/L$** ; the S-matrix algorithm then keeps the result stable. On a curved profile it is a staircase approximation. That remains acceptable in TE with many

slices, but converges very slowly in TM because the staircase never has the right normal [Popov 2002; Arnaud 2008 § 2.4]. For binary structures in TM, RK4 uses the finite-difference normal of the vertical walls, which converges more slowly in N than `Z_INVAR` ; prefer `Z_INVAR` there.

- `IMPROVED_RCWA` (md2D) uses z-invariant propagation with the true normal field of the profile.

## Cost

With OpenBLAS on an 8-core PC: md2D at  $N = 40$ ,  $NS = 1000$  takes 4 s in TE and 11 s in TM; md3D (pyramid,  $NS = 20$ ) takes 4 s at  $N = 6$  and about 1.5 min at  $N = 10$ . The programs choose the number of OpenBLAS threads themselves (1 for small matrices, 4 for large ones) unless `OPENBLAS_NUM_THREADS` is set. The reference BLAS is 3–8 times slower.

## Recommended routine

1. Start with AUTO or modest values and check that  $1 - \Sigma e$  is small for a lossless structure.
2. Double NS until the efficiencies you need no longer change at the wanted precision.
3. Increase N by 30–50 % and check again. Report the pair (N, NS) with the results.

## 8 Limitations and known issues

---

- **Isotropic, non-magnetic, linear media only** ( $\mu = \mu_0$ ). Only plane-wave illumination (`PLANE_WAVE`) is implemented.
- **md2D is for classical incidence only** ( $\varphi = 0$ ). For conical incidence use md3D with `Ny = 0`.
- `delta_h` is required but unused in both programs.
- **The shooting method** of the thesis (Table 2.1) is disabled; the matrix RK4 scheme replaces it and gives the same results.
- **Normal fields** are automatic only for single surfaces and stacks of interfaces. md2D `N_XYZ` has none (TE only). md3D `N_XY_ZINVAR` uses a hard-coded radial field centred on the cell (`Normal_N_XY_ZINVAR` in `md3D.c`); the code that would read `norm_x / norm_y` from the structure file is never reached. Any other shape needs a code change.
- **Experimental:** md3D `N_XYZ`, `H_XY_plus_STACK` (a sample run violates energy conservation), md2D `GUIDED` / `SWIFTS` modes and the `IMPLICIT_RK` / `EULER` schemes.
- **3D size:** the cost grows as the cube of the number of harmonics. Structures larger than a few wavelengths per period need an optimised BLAS and patience.
- Source comments are mostly in French (files are UTF-8).

## 9 Validation

---

The thirteen validation tables of the thesis were recomputed with the current code in September 2026. Nine are reproduced within the precision the thesis claims, one agrees to 0.01–0.3 % (conical incidence, originally computed with a code that no longer exists), and two could be checked only in part (the shooting-method column and absolute timings). The comparisons cover the method of fictitious sources, the integral method, RCWA, the Chandezon method, the method of variation of boundaries, GSolver, Schuster et al., energy conservation and reciprocity. The deep-aluminium table (4.6) is still to be run.

- Validation review: case-by-case results, charts and the remaining work.
- `REPRODUCING.md` and one `run.sh` per case in `validation_cases/`. Each case is a complete, runnable example of a parameter file and a structure.
- Thesis test cases: the original tables and their parameters.

## Automated tests

From the repository root: `make test` runs about 100 fast checks in under a minute. They compare with stored reference results, check physical laws (energy conservation, reciprocity, symmetries), compare with exact solutions (Fresnel, thin-film Airy formula), check md2D against md3D, and test input parsing and error handling. `make test-validation` recomputes every thesis table at reduced resolution against the published values (about 1.5 min), and `make bench` times a fixed set of runs. See `tests/README.md`.

## 10 References

---

1. [Arnaud 2008] L. Arnaud, *Diffraction et diffusion de la lumière : modélisation tridimensionnelle et application à la métrologie de la microélectronique et aux techniques d'imagerie sélective en milieu diffusant*, PhD thesis, Institut Fresnel (Marseille) and STMicroelectronics (Rousset), defended 17 July 2008. In French.
2. [Nevière & Popov 2003] M. Nevière, E. Popov, *Light Propagation in Periodic Media: Differential Theory and Design*, Marcel Dekker, 2003. The reference text for the differential method.
3. [Li 1996a] L. Li, "Use of Fourier series in the analysis of discontinuous periodic structures," *JOSA A* 13, 1870 (1996). doi:10.1364/JOSAA.13.001870
4. [Li 1996b] L. Li, "Formulation and comparison of two recursive matrix algorithms for modeling layered diffraction gratings," *JOSA A* 13, 1024 (1996). doi:10.1364/JOSAA.13.001024
5. [Popov & Nevière 2000] E. Popov, M. Nevière, "Grating theory: new equations in Fourier space leading to fast converging results for TM polarization," *JOSA A* 17, 1773 (2000). doi:10.1364/JOSAA.17.001773
6. [Popov & Nevière 2001] E. Popov, M. Nevière, "Maxwell equations in Fourier space: fast-converging formulation for diffraction by arbitrary shaped, periodic, anisotropic media," *JOSA A* 18, 2886 (2001). doi:10.1364/JOSAA.18.002886
7. [Popov 2002] E. Popov, M. Nevière, B. Gralak, G. Tayeb, "Staircase approximation validity for arbitrary-shaped gratings," *JOSA A* 19, 33 (2002). doi:10.1364/JOSAA.19.000033
8. [Li 1997] L. Li, "New formulation of the Fourier modal method for crossed surface-relief gratings," *JOSA A* 14, 2758 (1997). doi:10.1364/JOSAA.14.002758
9. [Schuster 2007] T. Schuster, J. Ruoff, N. Kerwien, S. Rafler, W. Osten, "Normal vector method for convergence improvement using the RCWA for crossed gratings," *JOSA A* 24, 2880 (2007). doi:10.1364/JOSAA.24.002880
10. [Moharam 1995] M. G. Moharam, E. B. Grann, D. A. Pommet, T. K. Gaylord, "Formulation for stable and efficient implementation of the rigorous coupled-wave analysis of binary gratings," *JOSA A* 12, 1068 (1995). doi:10.1364/JOSAA.12.001068
11. [Lalanne 1996] P. Lalanne, G. M. Morris, "Highly improved convergence of the coupled-wave method for TM polarization," *JOSA A* 13, 779 (1996). doi:10.1364/JOSAA.13.000779 · [Granet 1996] G. Granet, B. Guizal, *JOSA A* 13, 1019 (1996). doi:10.1364/JOSAA.13.001019
12. [Tayeb 1994] G. Tayeb, "The method of fictitious sources applied to diffraction gratings," *ACES Journal* 9, 90 (1994). Reference values used in validation cases 03–05.